

Competitive Programming

Section 42 Editorial: Advanced Data Structures Mixed Contest

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Dynamic Order Statistics	5
2.1	Restatement	5
2.2	Observations, trace, and algorithm	5
2.3	Correctness proof and complexity	5
2.4	Full C++ solution	5
2.5	Full Python solution	5
2.6	Tests that matter	6
3	B. Range Add Range Sum	7
3.1	Restatement	7
3.2	Observations, trace, and algorithm	7
3.3	Correctness proof and complexity	7
3.4	Full C++ solution	7
3.5	Full Python solution	7
3.6	Tests that matter	8
4	C. Static Rectangle Count	9
4.1	Restatement	9
4.2	Observations, trace, and algorithm	9
4.3	Correctness proof and complexity	9
4.4	Full C++ solution	9
4.5	Full Python solution	9
4.6	Tests that matter	10
5	What to carry forward	11
6	Testing discussion	12
7	What to carry forward	13

1 Editorial overview

1.1 Problem-specific proof map

A. Dynamic Order Statistics. A Fenwick prefix is exactly the number of stored values at most that index. Fenwick binary lifting finds the first prefix whose count reaches k , hence the k -th value, in $O(\log m)$ per operation.

B. Range Add Range Sum. The two-tree formula is the prefix sum of the difference array after distributing each range update over its affected positions. Subtracting two such prefixes therefore gives exactly the requested range sum, with $O(\log n)$ updates and queries.

C. Static Rectangle Count. Inclusion-exclusion turns a rectangle into four dominance counts. During the x -sweep, the Fenwick tree contains precisely points with $x \leq X$, so its y -prefix produces each dominance count in $O(\log n)$.

1.2 Detailed solution guide

A. Dynamic Order Statistics. A Fenwick tree over value coordinates stores frequencies. $\text{sum}(x)$ is the number of elements $\leq x$. Insert and erase update one coordinate by $+1$ or -1 . The k -th element is the smallest coordinate whose prefix sum is at least k ; binary lifting inside the Fenwick tree finds that coordinate in logarithmic time.

B. Range Add, Range Sum. Difference arrays turn a range add into two point events, but range-sum queries need one more accumulated moment. Maintain two Fenwick trees $B1$ and $B2$ so that after adding delta to $[l, r]$, $\text{prefix}(p) = p \cdot \text{sum}(B1, p) - \text{sum}(B2, p)$. Then $\text{sum}(l, r)$ is $\text{prefix}(r) - \text{prefix}(l-1)$.

C. Offline Rectangle Counting. Count points in rectangle $[x1, x2] \times [y1, y2]$ by inclusion-exclusion of four dominance queries $\text{count}(x \leq X, y \leq Y)$. Sort points and queries by X , sweep from left to right, and insert active point y -coordinates into a Fenwick tree. The y -prefix sum then answers the dominance query.

Common pitfalls. Coordinate-compress values before using a Fenwick tree when coordinates are large. For range sums, update both trees with matching signs. For rectangles, inclusion-exclusion signs are $+$ $-$ $-$ $+$.

All three problems reduce a seemingly richer query to prefix queries.

- A stores frequencies in a Fenwick tree. Counts are prefix sums; k -th order statistic is the first index whose prefix sum reaches k .
- B stores a range-added array through two Fenwick trees. The formula $\text{prefix}(p) = p \cdot \text{sum}(B1, p) - \text{sum}(B2, p)$ converts range additions into prefix sums of the final array.
- C expands each rectangle into four dominance queries $x \leq X, y \leq Y$, then sweeps points by x -coordinate and answers y -prefix counts with a Fenwick tree.

The recurring invariant is that the Fenwick tree contains exactly the active objects for the current prefix threshold.

2 A. Dynamic Order Statistics

2.1 Restatement

Maintain a multiset over values $1..m$: insert, erase one copy, k-th, and count $\leq x$.

2.2 Observations, trace, and algorithm

Keep multiplicities in a Fenwick tree. After inserts 3,5,3, rank two is 3 and count ≤ 4 is two; after erasing one 3, rank two is 5. Insert/erase update one index, count is a prefix sum, and kth is the first prefix reaching k by Fenwick binary lifting; reject an invalid k.

2.3 Correctness proof and complexity

Fenwick prefixes equal cumulative multiplicities, so the first prefix reaching k is exactly the kth multiset value. Every operation is $O(\log m)$ and memory is $O(m)$. QED.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int> b; BIT(int n):n(n),b(n+1){} void add(int
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int m,q;if(!(cin>>m>>q)) return 0; E
```

2.5 Full Python solution

```
import sys
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
    def kth(self,k):
        if k<1 or k>self.sum(self.n): return -1
        pos=0; step=1<<self.n.bit_length()
```

```

        while step:
            nxt=pos+step
            if nxt<=self.n and self.bit[nxt]<k: pos=nxt; k-=self.bit[nxt]
            step//=2
        return pos+1
def main():
    data=list(map(int,sys.stdin.buffer.read().split()));
    if not data: return
    m,q=data[0],data[1]; bit=BIT(m); cnt=[0]*(m+1); idx=2; out=[]
    for _ in range(q):
        t,x=data[idx],data[idx+1]; idx+=2
        if t==1: cnt[x]+=1; bit.add(x,1)
        elif t==2:
            if cnt[x]: cnt[x]-=1; bit.add(x,-1)
        elif t==3: out.append(str(bit.kth(x)))
        else: out.append(str(bit.sum(x)))
    print("\n".join(out))
if __name__=="__main__": main()

```

2.6 Tests that matter

Test duplicate insert/erase and invalid ranks, negative/full-range updates, and points on rectangle boundaries with duplicate coordinates.

3 B. Range Add Range Sum

3.1 Restatement

Support range addition and range-sum queries.

3.2 Observations, trace, and algorithm

An add x on $[l, r]$ writes $+x$ at l and $-x$ at $r+1$ in a difference tree. A second weighted tree gives $\text{prefix}(i) = i * \text{sum}(B1, i) - \text{sum}(B2, i)$. Adding ten on $[2, 4]$ makes the added contribution through $i=3$ equal $3*10-10=20$.

3.3 Correctness proof and complexity

Summing $B1$ reconstructs each pending add; summing those reconstructed values gives the weighted formula. Prefix subtraction isolates a range. Updates and queries are $O(\log n)$, memory $O(n)$. QED.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<long long>b; BIT(int n):n(n),b(n+2){} void add
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0; E
```

3.5 Full Python solution

```
import sys
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+2)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
```

```

if not data: return
n,q=data[0],data[1]; a=data[2:2+n]; b1=BIT(n+1); b2=BIT(n+1)
def add_range(l,r,x): b1.add(l,x); b1.add(r+1,-x); b2.add(l,x*(l-1)); b2.add(r+1,-x*r)
def pref(i): return b1.sum(i)*i-b2.sum(i)
for i,x in enumerate(a,1): add_range(i,i,x)
idx=2+n; out=[]
for _ in range(q):
    t=data[idx]; idx+=1
    if t==1:
        l,r,x=data[idx],data[idx+1],data[idx+2]; idx+=3; add_range(l,r,x)
    else:
        l,r=data[idx],data[idx+1]; idx+=2; out.append(str(pref(r)-pref(l-1)))
print("\n".join(out))
if __name__=="__main__": main()

```

3.6 Tests that matter

Test duplicate insert/erase, invalid ranks, negative/full-range updates, and points on inclusive rectangle boundaries with duplicate coordinates.

4 C. Static Rectangle Count

4.1 Restatement

Given points and rectangle queries, count points inside each rectangle.

4.2 Observations, trace, and algorithm

Turn a rectangle into x-prefix events at x_2 (+) and x_1-1 (-). Sort events and points by x; a compressed-y Fenwick tree contains points already passed. For points (1,1),(2,3),(4,2), rectangle [1,3]x[1,3] sees two points at $x=3$ and zero at $x=0$.

4.3 Correctness proof and complexity

At an event, the tree contains exactly points with x at most its threshold; its y-prefix difference counts the requested y interval. Subtracting events is rectangle inclusion-exclusion. Cost is $O((n+q)\log(n+q))$, memory $O(n+q)$. QED.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct BIT{int n; vector<int>b; BIT(int n):n(n),b(n+1){} void add(int i, int x){while(i<=n){b[i]+=x; i+=i&-i}}
struct E{int x,y1,y2,id,sgn; bool operator<(const E&o)const{return x<o.x;}};
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,q;if(!(cin>>n>>q)) return 0;v
```

4.5 Full Python solution

```
import sys,bisect
class BIT:
    def __init__(self,n): self.n=n; self.bit=[0]*(n+1)
    def add(self,i,x):
        while i<=self.n: self.bit[i]+=x; i+=i&-i
    def sum(self,i):
        s=0
        while i>0: s+=self.bit[i]; i-=i&-i
        return s
def main():
```

```

data=list(map(int,sys.stdin.buffer.read().split()))
if not data: return
n,q=data[0],data[1]; idx=2; pts=[]; ys=[]
for _ in range(n): x,y=data[idx],data[idx+1]; idx+=2; pts.append((x,y)); ys.append(y)
events=[]; ans=[0]*q
for qi in range(q):
    x1,y1,x2,y2=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; ys+= [y1,y2]
    events.append((x2,y1,y2,qi,1)); events.append((x1-1,y1,y2,qi,-1))
ys=sorted(set(ys)); pts.sort(); events.sort(); bit=BIT(len(ys)); p=0
for x,y1,y2,qi,sgn in events:
    while p<n and pts[p][0]<=x:
        bit.add(bisect.bisect_left(ys,pts[p][1])+1,1); p+=1
    l=bisect.bisect_left(ys,y1)+1; r=bisect.bisect_right(ys,y2)
    ans[qi]+=sgn*(bit.sum(r)-bit.sum(l-1))
print("\n".join(map(str,ans)))
if __name__=="__main__": main()

```

4.6 Tests that matter

Test duplicate insert/erase, invalid ranks, negative/full-range updates, and points on inclusive rectangle boundaries with duplicate coordinates.

5 What to carry forward

6 Testing discussion

Run `CP_TARGET=solution python3 sections/42_advanced_data_structures_mixed/check.py`. For small random cases, compare A with a sorted multiset, B with a direct array, and C with a direct point scan. These oracles expose incorrect binary lifting, off-by-one difference boundaries, and compression errors.

7 What to carry forward

Fenwick trees are prefix-summary engines. Pair them when a range update needs an extra weighted correction, binary-lift them when a prefix count becomes an order statistic, and combine one with a sweep when one coordinate can be made monotone.

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.